INCIDENT EVIDENCE WALKTHROUGH

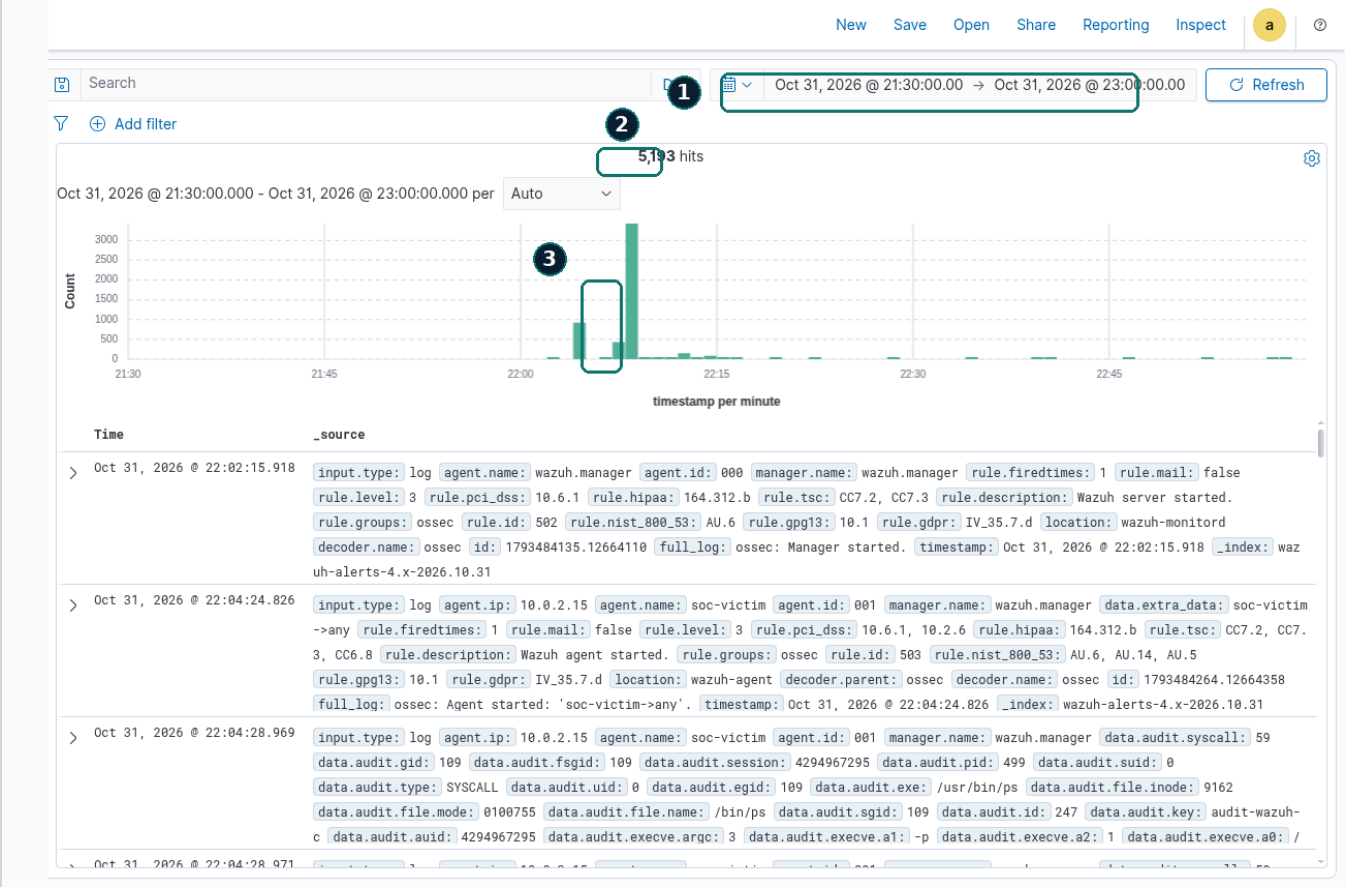
Web shell executed as www-data

INC-003 · High severity · 31 October 2026 · soc-victim

3	1	33	11 s
shell requests	PHP file added	www-data uid	upload to execution

1. Start with the evening

Ninety minutes around the web activity. 573 alerts, most of them audit noise. The web block at 22:12 is small and easy to miss.



1. Full window in UTC, empty query.
2. 573 alerts. The tallest bars are routine SSH and audit activity.
3. The 22:12 cluster is the upload and the three shell requests. It is small next to the noise.

WHAT TO LOOK FOR IN THE LOGS

Web alerts are rare

Most of the window is sshd and audit. Any web rule stands out by rule group, not volume.

Look for 80790

A file created under /var/www/html is the upload.

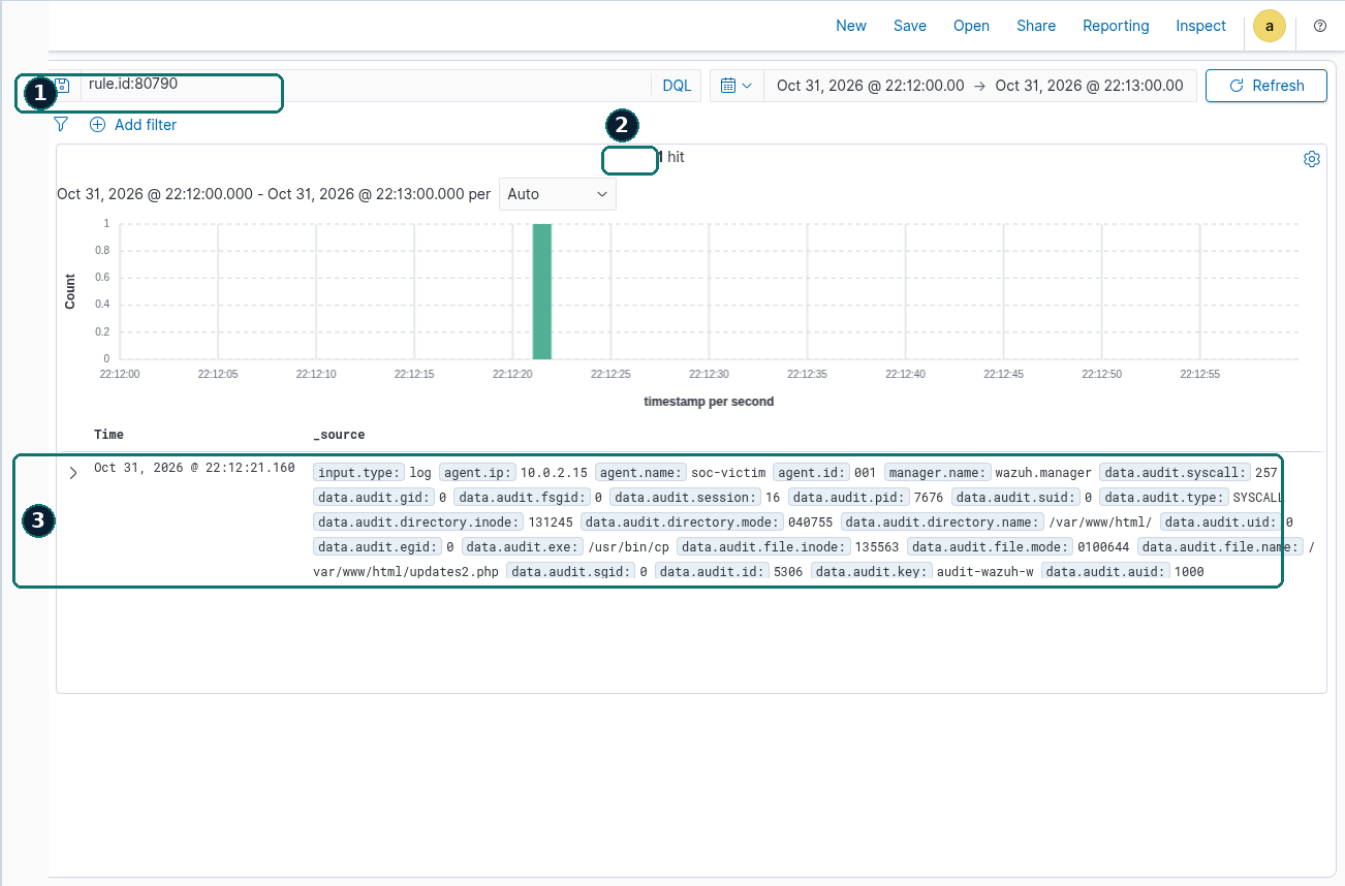
Correlate to the request

File creation at 22:12:21, requests at 22:12:24 and later.

> This is why the next step starts from the file event rather than the chart.

2. Stock rules see the file being created

Rule 80790 records a file created under a watched path. It is stock Wazuh, no custom rule involved.



1. Query on the stock audit rule.
2. One hit at 22:12:21.
3. The row names the file and the command that created it: sudo cp /tmp/updates2.php /var/www/html/updates2.php.

WHAT TO LOOK FOR IN THE LOGS

Path

/var/www/html is the webroot. A new PHP file there is a code change.

Command

data.audit.command shows the copy, and the sudo parent shows who ran it.

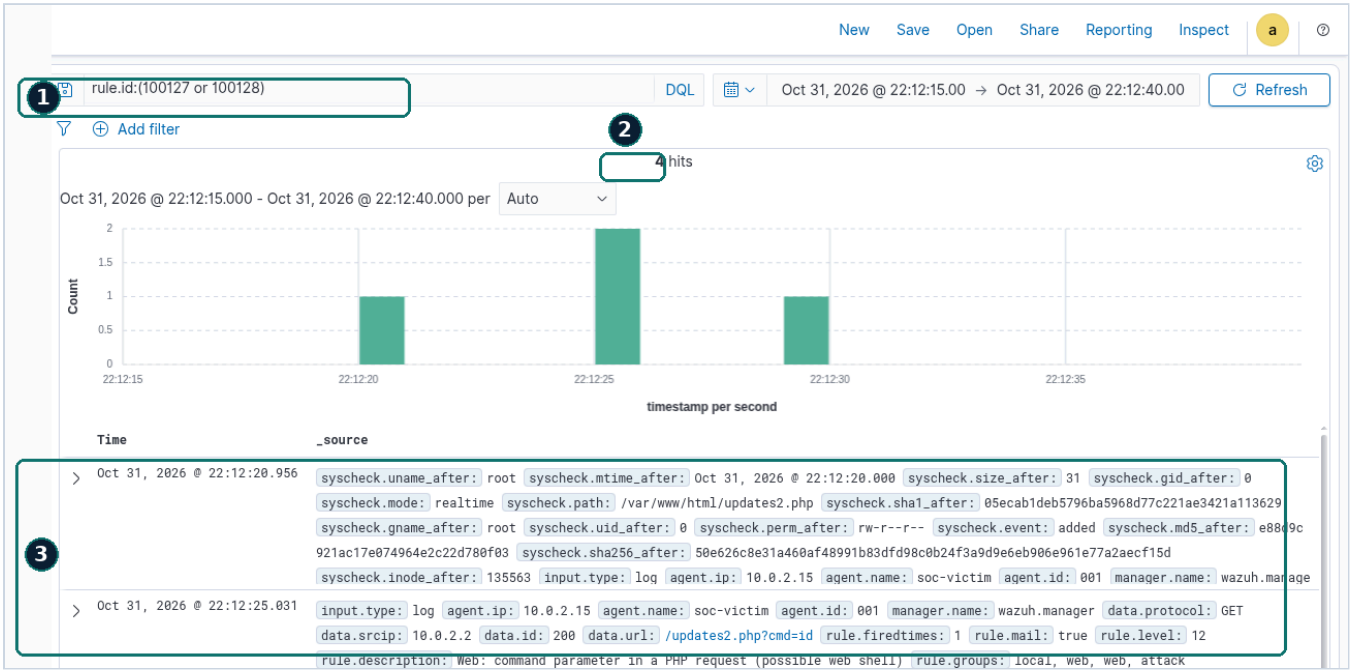
Time

22:12:20, four seconds before the first shell request.

> Stock rules say a file appeared. They do not say it is executable evidence of a shell.

3. Apply the custom rules

Two rules cover the request pattern and the file type. 100127 matches a command parameter in a PHP request, 100128 matches a PHP file added to a watched path.



1. Query on the two custom rules.
2. Four hits: one file event and three requests.
3. The web rows show cmd=id and cmd=uname, all returning 200.

WHAT TO LOOK FOR IN THE LOGS

Parameter name

cmd= in the query string of a .php request.

Response

HTTP 200 with a short body, consistent with command output.

Repetition

Three requests in four seconds from curl and sqlmap user agents.

File type

A .php file added under the webroot, one second before the first request.

> The rule matches the request, not the response body. The access log and the audit trail provide the impact.

4. The web shell timeline

Five events, exported in order. The file arrives first, the requests follow.

soc-lab, INC-003, alert chain, UTC

```
$ jq -r '.[] | select(.rule.id=="80790" or .rule.id=="100127"
      or .rule.id=="100128")
      | "\(.timestamp[11:19])  \(.rule.id)  \(.rule.description)"' \
      docs/evidence/INC-003-alerts.json
22:12:20  100128  FIM: PHP file added under a monitored path (possible web shell)
22:12:21  80790  Audit: Created: /var/www/html/updates2.php.
22:12:25  100127  Web: command parameter in a PHP request (possible web shell)
22:12:25  100127  Web: command parameter in a PHP request (possible web shell)
22:12:29  100127  Web: command parameter in a PHP request (possible web shell)
```

1. 100128 at 22:12:20: the PHP file appears in the webroot.
2. 80790 at 22:12:21: audit confirms the file was created.
3. 100127 at 22:12:25 and 22:12:29: shell requests with cmd=id and cmd=uname.

ALERT MIX

RULE	ALERTS	WHAT IT ADDS
100128	1	PHP file added to the webroot
80790	1	Audit record of the file creation
100127	3	Requests with a cmd parameter

> The order matters. A request with no preceding upload is a different case, and a file with no requests is a different severity.

5. Request evidence

The nginx access log is the clearest proof that the web shell executed.

soc-lab, INC-003, nginx access log

```
$ grep '22:12' docs/evidence/INC-003-web.log
10.0.2.2 - - [31/Oct/2026:22:12:24 +0000] "GET /updates2.php?cmd=id HTTP/1.1" 200 65 "-" "curl/8.22.0"
10.0.2.2 - - [31/Oct/2026:22:12:24 +0000] "GET /updates2.php?cmd=uname%20-a HTTP/1.1" 200 119 "-" "curl/8.22.0"
10.0.2.2 - - [31/Oct/2026:22:12:28 +0000] "GET /updates2.php?cmd=id HTTP/1.1" 200 65 "-" "sqlmap/1.7"
```

1. GET /updates2.php?cmd=id returns 200 with 65 bytes, the size of a short id output.
2. A second request asks for uname, and a third repeats id with sqlmap as the client.

WHAT TO LOOK FOR IN THE LOGS

Status and size

200 with a small body is command output, not an error page.

Client

curl and sqlmap user agents. Neither belongs in normal browsing.

Path

A file that did not exist before 22:12:20.

> Save the log lines and the file hash. The response body is not in the log, so record what the request returned if you can reproduce it.

6. Upload and execution trail

auditd shows the copy into the webroot and the shell that followed.

soc-lab, INC-003, upload and execution

```
$ grep -E 'sh -c id|sha256sum|updates2.php' \
    docs/evidence/INC-003-commands.txt | head -8
1 22:12:20Z  sudo cp /tmp/updates2.php /var/www/html/updates2.php
  22:12:20Z  cp /tmp/updates2.php /var/www/html/updates2.php
  22:12:20Z  sudo sha256sum /var/www/html/updates2.php
2 22:12:20Z  sha256sum /var/www/html/updates2.php
  22:12:24Z  sh -c id
  22:12:24Z  id
  22:12:28Z  sh -c id
```

1. The hex payload decodes to cp, chmod-style move, and sha256sum against the webroot file.
2. sh -c id runs at 22:12:24, confirming command execution through the web server.

WHAT TO LOOK FOR IN THE LOGS

Who wrote the file

The sudo cp is run by the analyst account, not by the web server.

Who executed the command

The request is served by www-data, uid 33.

Hash

Record the PHP file hash before removal.

> Upload and execution are two facts. A ticket that only says a file appeared misses the execution.

7. How to capture this evidence

1. **Start broad**
   Ninety minutes around the event, empty query. Note how small the web cluster is.

2. **Stock rule**
   The file creation event.

```
rule.id:80790
```

3. **Custom rules**
   Request pattern and PHP file.

```
rule.id:(100127 or 100128)
```

4. **Save the access log**
   The request lines are the proof of execution.

```
grep 'cmd=' /var/log/nginx/access.log
```

5. **Record the SHA-256 before you delete the file**
   The command only reads the file. It does not execute or change it.

```
ssh -i keys/id_ed25519 -p 2222 analyst@127.0.0.1 \
  'sudo sha256sum /var/www/html/updates2.php'
```

6. **Export the alerts**
   JSON as evidence, screenshot as context.

```
curl -sk -u admin:"$WAZUH_PASS" \
  'https://localhost:9200/wazuh-alerts-*/_search?size=500' \
  -d '{"query":{"range":{"@timestamp":{"gte":"2026-10-31T22:10:00Z","lte":"2026-10-
31T22:20:00Z"}}}}' \
  | jq '.hits.hits[]._source' > INC-003-alerts.json
```

> A web shell is proven by the request log plus the process trail. The alert alone does not show execution.

8. What to look for

Command parameter in the query
cmd=, exec=, or a similar name on a PHP endpoint.

A new PHP file
The upload precedes the requests by seconds.

200 with a small body
The response size often matches command output.

Client user agents
curl, sqlmap, or a browser string that does not fit the asset.

Process identity
www-data executing sh or id is the execution proof.

COMMON PITFALLS

One rule is not enough
100127 matches requests, 100128 matches files. Together they tell the story.

Response body missing
Access logs do not store the body. Note the size and reproduce carefully.

Path lookalikes
A harmless phpinfo page also returns 200. Read the parameter and the process trail.

Verdict: High severity, contained. The shell was removed, the webroot restored, and the detection rule kept for future requests. No persistence was found.